Sentiment Analysis with Naive Bayes

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable ~~with~~ no fun

1. Calculate the priors

$$P(-) = \frac{3}{5} \qquad P(+) = \frac{2}{5}$$

2. Drop "with"

CC BY-NC-SA

Sentiment Analysis with Naive Bayes

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable ~~with~~ no fun

1. Calculate the priors

$$P(-) = \frac{3}{5} \qquad P(+) = \frac{2}{5}$$

2. Drop "with"

3. Calculate the likelihoods

$$P(\text{“predictable”}|-) = \frac{1+1}{14+20} \qquad P(\text{“predictable”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“no”}|-) = \frac{1+1}{14+20} \qquad P(\text{“no”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“fun”}|-) = \frac{0+1}{14+20} \qquad P(\text{“fun”}|+) = \frac{1+1}{9+20}$$

4. Predict the test sentence

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

- Based on the training data identify the class of the test case
- The word "with" in test set doesn't exist in training set
    - Drop case
- For each word in test case, calculate the probability of appearance for all cases
    - Include laplace smoothing for each probability calculations
    - Denominator = total of word in class + total number of words in entire document
    - Then multiply class probability with each word probability
- Combination of probability
    - The returned higher probability of all cases would return the class of the test case

Optimizing for Sentiment Analysis

For sentiment classification and a number of other text classification tasks, whether a word occurs or not seems to matter more than its frequency

Thus it often improves performance to clip the word counts in each document at 1, this is called **Binary Naive Bayes**

The variant uses the *same algorithm* as earlier except that for each document

- we remove all duplicate words before concatenating them into the single big document during training and
- we also remove duplicate words from test documents

CC BY-NC-SA

Optimizing for Sentiment Analysis

Four original documents:

- − it was pathetic the worst part was the boxing scenes
- − no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- − it was pathetic the worst part boxing scenes
- − no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	−	+	−
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Figure 4.3 An example of binarization for the binary naive Bayes algorithm.

Remove duplicate occurrences for each line
Each line is independent
Only applied in documents not the class

Optimizing for Sentiment Analysis

A second important addition commonly made when doing text classification for sentiment is to deal with **negation**

Consider the difference between

- "I really like this movie" (positive) and
- "I didn't like this movie" (negative)

The negation expressed by didn't completely alters the inferences we draw from the predicate like

Thus the phrase "didn't like this movie" becomes "didn't NOT_like NOT_this NOT_movie"

- Applies when a word in one case that defines the class also appears in the other class but means the opposite

Optimizing for Sentiment Analysis

Finally, we might have **insufficient labeled training data** to train accurate classifiers using all words in the training set to estimate positive and negative sentiment.

We can instead derive the positive and negative word features from **sentiment lexicons**

- lists of words that are pre-annotated with positive or negative sentiment.

e.g. MPQA Subjectivity Cues Lexicon

- Home page: https://mpqa.cs.pitt.edu/lexicons/subj_lexicon/
- 6885 words from 8221 lemmas, annotated for intensity (strong/weak)
- 2718 positive
- 4912 negative
- + : admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great
- − : awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate

CC BY-NC-SA

Summary

Naive Bayes… not so naive!

CC BY-NC-SA

Summary

Very Fast, low storage requirements

Work well with very small amounts of training data

Robust to irrelevant Features

- irrelevant Features cancel each other without affecting results

Very good in domains with many equally important features

- Decision Trees suffer from fragmentation in such cases – especially if little data

Optimal if the independence assumptions hold:

- If assumed independence is correct, then it is the Bayes Optimal Classifier for problem

A good dependable baseline for text classification

- But we will see other classifiers that give better accuracy

CC BY-NC-SA